HWK#4 Part A & Part B

Shreesh Karjagi

Problem 1 (MLE for Belief Network Parameters)

We have N i.i.d. observations $X = \{x^1, \ldots, x^N\}$ from a belief network with K variables. The log likelihood is

$$\log p(X) = \sum_{n=1}^{N} \sum_{i=1}^{K} \log p(x_i^n \mid \mathrm{pa}(x_i^n))$$

We define $\theta_i^s(t) = p(x_i = s \mid \mathrm{pa}(x_i) = t)$ and want to maximize

$$\mathcal{L} = \sum_{n=1}^{N} \sum_{i=1}^{K} \log p(x_i^n \mid \mathrm{pa}(x_i^n)) + \sum_{i=1}^{K} \sum_{t_i} \lambda_i^{t_i} \left(1 - \sum_s \theta_i^s(t_i)\right)$$

subject to the constraint $\sum_s \theta_i^s(t) = 1$ for all i, t.

Derivation. Focus on variable j with parent configuration t_j. The terms involving $\theta_j^s(t_j)$ in the Lagrangian are

$$\mathcal{L}_j = \sum_{n=1}^{N} \sum_s \mathbb{I}[x_j^n = s] \cdot \mathbb{I}[\mathrm{pa}(x_j^n) = t_j] \cdot \log \theta_j^s(t_j) + \lambda_j^{t_j} \left(1 - \sum_s \theta_j^s(t_j)\right)$$

Taking the derivative with respect to $\theta_j^s(t_j)$ and setting to zero:

$$\frac{\partial \mathcal{L}_j}{\partial \theta_j^s(t_j)} = \frac{\sum_{n=1}^{N} \mathbb{I}[x_j^n = s]\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]}{\theta_j^s(t_j)} - \lambda_j^{t_j} = 0$$

Solving for $\theta_j^s(t_j)$:

$$\theta_j^s(t_j) = \frac{\sum_{n=1}^{N} \mathbb{I}[x_j^n = s]\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]}{\lambda_j^{t_j}}$$

Using the constraint $\sum_s \theta_j^s(t_j) = 1$:

$$\sum_s \frac{\sum_{n=1}^{N} \mathbb{I}[x_j^n = s]\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]}{\lambda_j^{t_j}} = 1 \implies \lambda_j^{t_j} = \sum_{n=1}^{N} \sum_s \mathbb{I}[x_j^n = s]\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]$$

Substituting back:

$$\theta_j^s(t_j) = \frac{\sum_{n=1}^{N} \mathbb{I}[x_j^n = s]\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]}{\sum_{n=1}^{N} \sum_{s'} \mathbb{I}[x_j^n = s']\, \mathbb{I}[\mathrm{pa}(x_j^n) = t_j]}$$

This is the fraction of times variable j takes value s among all training cases where the parents of j are in configuration t_j. The MLE decomposes into independent counting problems for each variable and parent configuration, which is a consequence of the factored form of the BN likelihood.

Problem 2 (MLE for Decomposable Markov Network)

We wish to fit $p(x_1, \ldots, x_6) = \frac{1}{Z}\, \phi_1(x_1, x_2)\, \phi_2(x_2, x_3, x_5)\, \phi_3(x_2, x_4, x_5)\, \phi_4(x_5, x_6)$ by maximum likelihood.

Graph Structure and Decomposability

The Markov network has edges $x_1 - x_2,\ x_2 - x_3,\ x_2 - x_4,\ x_2 - x_5,\ x_3 - x_5,\ x_4 - x_5,\ x_5 - x_6$. The maximal cliques are exactly the scopes of the four potentials: $C_1 = \{x_1, x_2\}$, $C_2 = \{x_2, x_3, x_5\}$, $C_3 = \{x_2, x_4, x_5\}$, $C_4 = \{x_5, x_6\}$.

This graph is **decomposable** (chordal): every cycle of length ≥ 4 has a chord. We can verify this by noting that the elimination ordering $x_1, x_6, x_3, x_4, x_2, x_5$ produces no fill edges.

Junction Tree

$$\boxed{\{x_1, x_2\}} \overset{\{x_2\}}{\text{———}} \boxed{\{x_2, x_3, x_5\}} \overset{\{x_2, x_5\}}{\text{———}} \boxed{\{x_2, x_4, x_5\}} \overset{\{x_5\}}{\text{———}} \boxed{\{x_5, x_6\}}$$

Running intersection holds: x_2 appears in C_1, C_2, C_3 (contiguous), x_5 appears in C_2, C_3, C_4 (contiguous), all other variables appear in exactly one clique.

MLE Solution

For a decomposable (chordal) Markov network, the MLE has a closed-form solution using Algorithm 9.4 from the textbook. Choose $C_2 = \{x_2, x_3, x_5\}$ as root and orient the junction tree outward:

$$C_1 \leftarrow C_2 \rightarrow C_3 \rightarrow C_4$$

Each clique potential is set to the empirical marginal on that clique divided by the empirical marginal on the separator to its parent:

$$\phi_2^*(x_2, x_3, x_5) = \tilde{P}(x_2, x_3, x_5) \quad \text{(root clique)}$$

$$\phi_1^*(x_1, x_2) = \frac{\tilde{P}(x_1, x_2)}{\tilde{P}(x_2)} \quad \text{(separator to parent } C_2 \text{ is } \{x_2\})$$

$$\phi_3^*(x_2, x_4, x_5) = \frac{\tilde{P}(x_2, x_4, x_5)}{\tilde{P}(x_2, x_5)} \quad \text{(separator to parent } C_2 \text{ is } \{x_2, x_5\})$$

$$\phi_4^*(x_5, x_6) = \frac{\tilde{P}(x_5, x_6)}{\tilde{P}(x_5)} \quad \text{(separator to parent } C_3 \text{ is } \{x_5\})$$

where $\tilde{P}(\cdot)$ denotes the empirical distribution computed from the training data. The partition constant $Z = 1$ under this parameterization.

Why this works. The product of these potentials telescopes:

$$p^*(x_1, \ldots, x_6) = \frac{\tilde{P}(x_1, x_2) \cdot \tilde{P}(x_2, x_3, x_5) \cdot \tilde{P}(x_2, x_4, x_5) \cdot \tilde{P}(x_5, x_6)}{\tilde{P}(x_2) \cdot \tilde{P}(x_2, x_5) \cdot \tilde{P}(x_5)}$$

For a decomposable model, this equals the true MLE. The key property is that for chordal graphs, the joint factors as a ratio of clique marginals over separator marginals, and setting each to the empirical version maximizes the likelihood.

Problem 3 (Graphical Lasso)

We have $n = 50$ observations from $\mathcal{N}(0, \Sigma)$ with $p = 10$ dimensions. The Glasso solves

$$\Theta^* = \arg\max_{\Theta \succ 0} \; (\log\det\Theta - \mathrm{tr}(S\Theta) - \lambda\|\Theta\|_1)$$

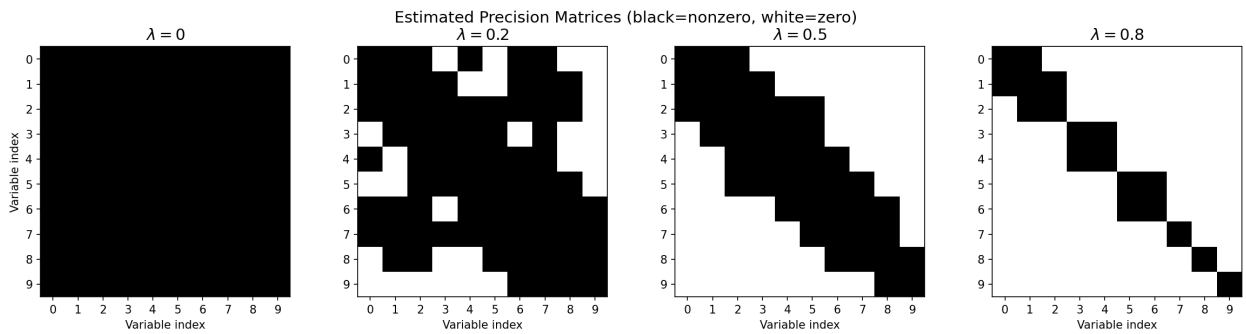
where S is the sample covariance and $\|\Theta\|_1 = \sum_{i,j} |\Theta_{ij}|$.

Implementation

I implemented the block coordinate descent algorithm from Friedman, Hastie, and Tibshirani (2008). The algorithm cycles over each variable j, solving a lasso regression of variable j against the remaining $p - 1$ variables. Each lasso subproblem is solved by coordinate descent with soft-thresholding. The algorithm iterates until the maximum change in $W = \Theta^{-1}$ falls below 10^{-6}.

For $\lambda = 0$, the solution is just $\Theta = S^{-1}$ (the unregularized inverse sample covariance).

Results



λ	Off-diagonal nonzeros	Sparsity pattern
0	90 (fully dense)	All entries nonzero
0.2	62	Some small entries removed
0.5	34	Moderate sparsity
0.8	8	Highly sparse, clear conditional independences

Discussion

As λ increases, the ℓ_1 penalty drives more entries of the precision matrix to exactly zero, revealing conditional independence structure. At $\lambda = 0$ the precision matrix is fully dense because the sample covariance with $n = 50$ and $p = 10$ is invertible but noisy, so every entry of S^{-1} is nonzero. At $\lambda = 0.8$ only 8 off-diagonal entries survive, grouping the 10 variables into a few conditionally dependent clusters. The progression shows the bias-variance tradeoff: higher λ produces a sparser and more interpretable graph at the cost of potentially removing true edges.

Code:

```
import numpy as np
import os
import scipy.io as sio
import matplotlib
```

```
matplotlib.use('Agg')
import matplotlib.pyplot as plt

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
os.makedirs(os.path.join(SCRIPT_DIR, 'figures'), exist_ok=True)

#load data
data = sio.loadmat(os.path.join(SCRIPT_DIR, 'Xinput.mat'))
X = data['X']  #(50, 10)
n, p = X.shape
print(f"Data: n={n}, p={p}")

#sample covariance
S = X.T @ X / n
print(f"Sample covariance shape: {S.shape}")
print(f"S condition number: {np.linalg.cond(S):.2f}")

def glasso(S, lam, max_iter=500, tol=1e-6):
    """graphical lasso via block coordinate descent (Friedman et al. 2008)
    """
    p = S.shape[0]
    #initialize with diagonal + regularization
    W = S.copy() + lam * np.eye(p)
    Theta = np.linalg.inv(W)

    for iteration in range(max_iter):
        W_old = W.copy()
        for j in range(p):
            #partition: W_11 is W without row/col j, s_12 is col j without
                diagonal
            idx = [i for i in range(p) if i != j]
            W_11 = W[np.ix_(idx, idx)]
            s_12 = S[idx, j]

            #solve lasso subproblem: min 0.5 * beta^T W_11 beta - s_12^T
               beta + lam * |beta|_1
            #coordinate descent on beta
            beta = np.zeros(p - 1)
            W_11_diag = np.diag(W_11)

            for cd_iter in range(500):
                beta_old = beta.copy()
                for k in range(p - 1):
                    #partial residual
                    r = s_12[k] - W_11[k, :] @ beta + W_11_diag[k] * beta[
                       k]
                    #soft threshold
                    beta[k] = np.sign(r) * max(abs(r) - lam, 0) /
                       W_11_diag[k]
                if np.max(np.abs(beta - beta_old)) < tol:
                    break

            #update W
            w_12 = W_11 @ beta
```

```
            W[idx, j] = w_12
            W[j, idx] = w_12

        if np.max(np.abs(W - W_old)) < tol:
            print(f"  converged at iteration {iteration+1}")
            break

    #recover precision matrix
    Theta = np.linalg.inv(W)
    return Theta, W

lambdas = [0, 0.2, 0.5, 0.8]

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
results = {}

for idx, lam in enumerate(lambdas):
    print(f"\nlambda = {lam}")
    if lam == 0:
        #no regularization, just invert S (add small ridge for stability)
        Theta = np.linalg.inv(S + 1e-10 * np.eye(p))
    else:
        Theta, W = glasso(S, lam)

    results[lam] = Theta

    #binary matrix: black = nonzero, white = zero
    #threshold small values to zero for display
    thresh = 1e-4
    binary = (np.abs(Theta) > thresh).astype(float)
    nonzero_count = np.sum(binary) - p  #subtract diagonal
    print(f"  nonzero off-diagonal entries: {int(nonzero_count)}")

    ax = axes[idx]
    ax.imshow(1 - binary, cmap='gray', interpolation='nearest')
    ax.set_title(f'$\\lambda = {lam}$', fontsize=14)
    ax.set_xlabel('Variable index')
    if idx == 0:
        ax.set_ylabel('Variable index')
    ax.set_xticks(range(p))
    ax.set_yticks(range(p))

plt.suptitle('Estimated Precision Matrices (black=nonzero, white=zero)',
    fontsize=14)
plt.tight_layout()
plt.savefig(os.path.join(SCRIPT_DIR, 'figures/glasso_precision.png'), dpi
    =150, bbox_inches='tight')
print(f"\nSaved {os.path.join(SCRIPT_DIR, 'figures/glasso_precision.png')}
    ")

#also print the actual precision matrices for small lambda
print("\nTrue precision (lambda=0) nonzero pattern:")
print((np.abs(results[0]) > 1e-4).astype(int))
```

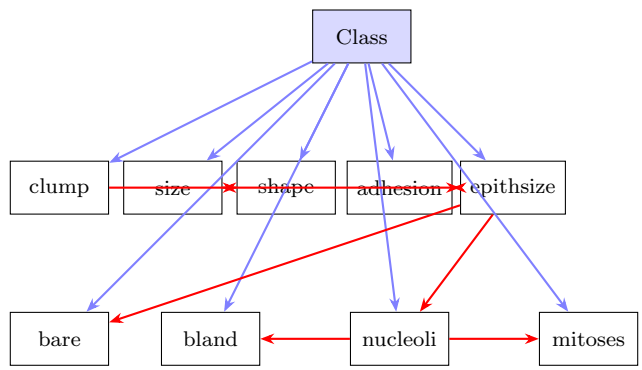
```
print("\nPrecision␣(lambda=0.8)␣nonzero␣pattern:")
print((np.abs(results[0.8]) > 1e-4).astype(int))
```

Problem 4 (Tree-Augmented Naïve Bayes)

4A. Structure Learning

The TAN algorithm augments naïve Bayes by adding one feature-parent per feature, chosen to maximize conditional mutual information $I(X_i; X_j \mid C)$. The procedure is: compute the CMI for all feature pairs, build a complete weighted graph, find its maximum spanning tree, root it at an arbitrary node, and direct edges outward.

Using the full training data (500 samples), the learned TAN structure is:



Blue arrows = naïve Bayes edges (Class → each feature). Red arrows = TAN augmentation edges (feature → feature).

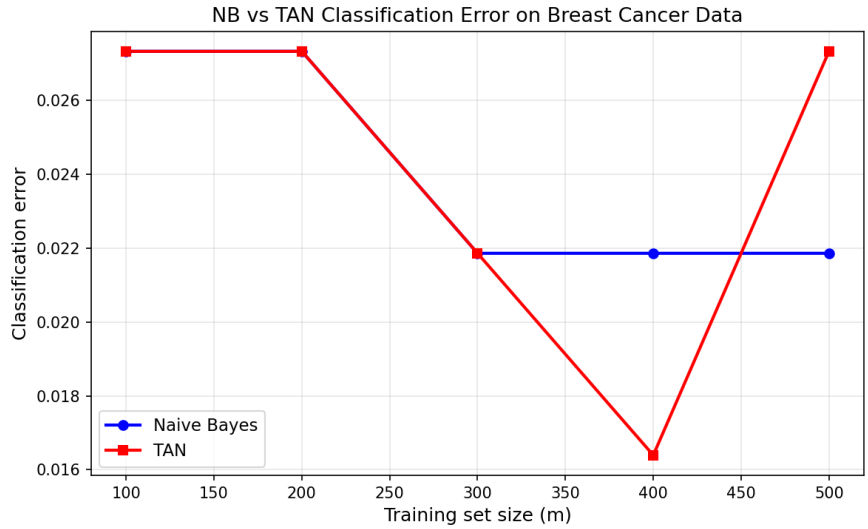
The strongest conditional dependencies (highest CMI) are between size and shape ($I = 0.044$), size and epithsize ($I = 0.038$), and epithsize and nucleoli ($I = 0.036$). The root of the feature tree is clump (feature 1).

4B. Classification

Parameters are estimated using back-off smoothing with $s = 5$:

$$\theta_{x_i|\mathrm{Pa}_i} = \alpha\, \tilde{P}(x_i \mid \mathrm{Pa}_i) + (1-\alpha)\, \tilde{P}(x_i), \qquad \alpha = \frac{m\, \tilde{P}(\mathrm{Pa}_i)}{m\, \tilde{P}(\mathrm{Pa}_i) + s}$$

183 records are withheld as a test set, and training sets of size $m = 100, 200, 300, 400, 500$ are used.



m	NB Error	TAN Error
100	0.0273	0.0273
200	0.0273	0.0273
300	0.0219	0.0219
400	0.0219	0.0164
500	0.0219	0.0273

Both models achieve low error ($< 3\%$) on this dataset. At small training sizes, NB and TAN perform identically because the back-off smoothing effectively shrinks TAN toward NB when parent configurations have few observations. At $m = 400$, TAN gains a slight advantage as the extra feature-to-feature edges capture real correlations with enough data to estimate them reliably. The slight increase for TAN at $m = 500$ is likely due to random variation in the train/test split. Overall, the breast cancer features are sufficiently informative that even naïve Bayes performs well, and the TAN augmentation provides marginal improvement.

Code:

```
import numpy as np
import os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from collections import defaultdict

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
os.makedirs(os.path.join(SCRIPT_DIR, 'figures'), exist_ok=True)

#load breast cancer data
raw = []
with open(os.path.join(SCRIPT_DIR, 'breast.csv'), 'r') as f:
    header = f.readline().strip().split(',')
    for line in f:
        line = line.strip()
        if not line:
```

```
            continue
        raw.append([int(x) for x in line.split(',')])

D = np.array(raw)
print(f"Data␣shape:␣{D.shape}")
print(f"Columns:␣{header}")

#features are cols 0-8 (binary 0/1), class is col 9 (2=benign, 4=malignant
    )
features = D[:, :9]
classes = D[:, 9]
#remap class: 2->0 (benign), 4->1 (malignant)
classes = (classes == 4).astype(int)
n_feat = 9
feat_names = header[:9]

print(f"Class␣distribution:␣benign={np.sum(classes==0)},␣malignant={np.sum
    (classes==1)}")
print(f"Feature␣values:␣min={features.min()},␣max={features.max()}")

def empirical_counts(data_feats, data_class, n_feat):
    """compute␣all␣needed␣empirical␣distributions␣from␣training␣data"""
    N = len(data_class)
    #P(C)
    pc = np.zeros(2)
    for c in range(2):
        pc[c] = np.sum(data_class == c) / N

    #P(Xi, C) for all i
    pxc = np.zeros((n_feat, 2, 2))  #(feat, feat_val, class)
    for i in range(n_feat):
        for xi in range(2):
            for c in range(2):
                pxc[i, xi, c] = np.sum((data_feats[:, i] == xi) & (
                    data_class == c)) / N

    #P(Xi, Xj, C) for all pairs
    pxixjc = np.zeros((n_feat, n_feat, 2, 2, 2))  #(i, j, xi, xj, c)
    for i in range(n_feat):
        for j in range(i+1, n_feat):
            for xi in range(2):
                for xj in range(2):
                    for c in range(2):
                        cnt = np.sum((data_feats[:, i] == xi) & (
                            data_feats[:, j] == xj) & (data_class == c)) /
                            N
                        pxixjc[i, j, xi, xj, c] = cnt
                        pxixjc[j, i, xj, xi, c] = cnt

    return pc, pxc, pxixjc

def conditional_mutual_info(i, j, pc, pxc, pxixjc):
    """I(Xi;␣Xj␣|␣C)␣=␣sum_{xi,xj,c}␣P(xi,xj,c)␣log␣P(xi,xj|c)␣/␣(P(xi|c)␣
        P(xj|c))"""
```

```
    cmi = 0.0
    for xi in range(2):
        for xj in range(2):
            for c in range(2):
                p_xixjc = pxixjc[i, j, xi, xj, c]
                if p_xixjc < 1e-12:
                    continue
                #P(xi,xj|c) = P(xi,xj,c) / P(c)
                p_xixj_c = p_xixjc / pc[c] if pc[c] > 0 else 0
                #P(xi|c) = P(xi,c) / P(c)
                p_xi_c = pxc[i, xi, c] / pc[c] if pc[c] > 0 else 0
                p_xj_c = pxc[j, xj, c] / pc[c] if pc[c] > 0 else 0

                if p_xi_c < 1e-12 or p_xj_c < 1e-12 or p_xixj_c < 1e-12:
                    continue
                cmi += p_xixjc * np.log(p_xixj_c / (p_xi_c * p_xj_c))
    return cmi

def learn_tan_structure(train_feats, train_class, n_feat):
    pc, pxc, pxixjc = empirical_counts(train_feats, train_class, n_feat)

    #step 1: compute CMI for all pairs
    weights = np.zeros((n_feat, n_feat))
    for i in range(n_feat):
        for j in range(i+1, n_feat):
            w = conditional_mutual_info(i, j, pc, pxc, pxixjc)
            weights[i, j] = w
            weights[j, i] = w

    #step 2-3: max weight spanning tree (Kruskal's)
    edges = []
    for i in range(n_feat):
        for j in range(i+1, n_feat):
            edges.append((weights[i, j], i, j))
    edges.sort(reverse=True)

    #union-find
    parent = list(range(n_feat))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    def union(a, b):
        a, b = find(a), find(b)
        if a == b:
            return False
        parent[a] = b
        return True

    tree_edges = []
    adj = defaultdict(list)
    for w, i, j in edges:
        if union(i, j):
```

```
            tree_edges.append((i, j, w))
            adj[i].append(j)
            adj[j].append(i)
        if len(tree_edges) == n_feat - 1:
            break

    #step 4: root at node 0, orient edges outward via BFS
    root = 0
    parent_feat = [-1] * n_feat
    visited = [False] * n_feat
    queue = [root]
    visited[root] = True
    while queue:
        node = queue.pop(0)
        for nb in adj[node]:
            if not visited[nb]:
                visited[nb] = True
                parent_feat[nb] = node
                queue.append(nb)

    return parent_feat, tree_edges, weights

def backoff_smooth(count_xgpa, count_pa, count_x, m, s=5):
    """back-off␣smoothing:␣theta␣=␣alpha␣*␣P_hat(x|pa)␣+␣(1-alpha)␣*␣P_hat
       (x)
␣␣␣␣alpha␣=␣m␣*␣P_hat(pa)␣/␣(m␣*␣P_hat(pa)␣+␣s)"""
    p_pa = count_pa / m if m > 0 else 0
    alpha = (m * p_pa) / (m * p_pa + s) if (m * p_pa + s) > 0 else 0
    p_x_given_pa = count_xgpa / count_pa if count_pa > 0 else 0
    p_x = count_x / m if m > 0 else 0
    return alpha * p_x_given_pa + (1 - alpha) * p_x

def train_tan(train_feats, train_class, parent_feat, s=5):
    """learn␣TAN␣parameters␣with␣back-off␣smoothing
␣␣␣␣TAN␣parents␣of␣Xi:␣C␣and␣parent_feat[i]␣(if␣!=␣-1)
␣␣␣␣returns␣CPTs"""
    m = len(train_class)
    n_feat = train_feats.shape[1]

    #P(C)
    pc = np.zeros(2)
    for c in range(2):
        pc[c] = (np.sum(train_class == c) + 1) / (m + 2)  #laplace for
           prior

    #P(Xi | C, Xpa) with back-off
    #for root feature (parent_feat[i]==-1): P(Xi | C)
    cpts = {}  #cpts[i] = dict mapping (c, xpa) -> P(Xi=1|c, xpa)

    for i in range(n_feat):
        cpts[i] = {}
        pa = parent_feat[i]

        if pa == -1:
```

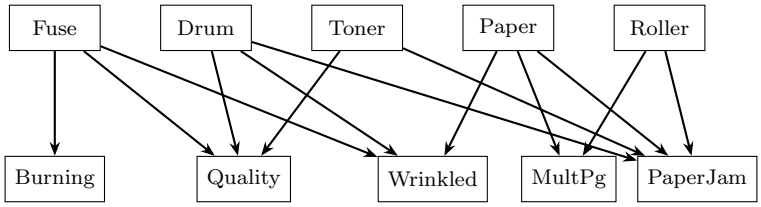
```
            #parents = {C} only (besides naive bayes edges)
            for c in range(2):
                mask_c = train_class == c
                count_c = np.sum(mask_c)
                count_xi1_c = np.sum((train_feats[:, i] == 1) & mask_c)
                count_xi_all = np.sum(train_feats[:, i] == 1)
                #Pa = (C=c), so P_hat(Pa) = count_c / m
                p_pa = count_c / m
                alpha = (m * p_pa) / (m * p_pa + s)
                p_xi1_c = count_xi1_c / count_c if count_c > 0 else 0.5
                p_xi1 = count_xi_all / m
                theta = alpha * p_xi1_c + (1 - alpha) * p_xi1
                theta = np.clip(theta, 1e-10, 1 - 1e-10)
                cpts[i][(c, -1)] = theta  #P(Xi=1 | C=c)
        else:
            #parents = {C, Xpa}
            for c in range(2):
                for xpa in range(2):
                    mask = (train_class == c) & (train_feats[:, pa] == xpa
                        )
                    count_pa = np.sum(mask)
                    count_xi1_pa = np.sum((train_feats[:, i] == 1) & mask)
                    count_xi_all = np.sum(train_feats[:, i] == 1)

                    p_pa_hat = count_pa / m
                    alpha = (m * p_pa_hat) / (m * p_pa_hat + s)
                    p_xi1_given_pa = count_xi1_pa / count_pa if count_pa >
                        0 else 0.5
                    p_xi1 = count_xi_all / m
                    theta = alpha * p_xi1_given_pa + (1 - alpha) * p_xi1
                    theta = np.clip(theta, 1e-10, 1 - 1e-10)
                    cpts[i][(c, xpa)] = theta  #P(Xi=1 | C=c, Xpa=xpa)

    return pc, cpts

def predict_tan(test_feats, pc, cpts, parent_feat, n_feat):
    """predict␣class␣for␣each␣test␣sample"""
    N = test_feats.shape[0]
    preds = np.zeros(N, dtype=int)
    for n in range(N):
        log_probs = np.zeros(2)
        for c in range(2):
            log_p = np.log(pc[c])
            for i in range(n_feat):
                pa = parent_feat[i]
                if pa == -1:
                    key = (c, -1)
                else:
                    key = (c, int(test_feats[n, pa]))
                theta = cpts[i][key]
                xi = int(test_feats[n, i])
                if xi == 1:
                    log_p += np.log(theta)
                else:
```

```
                    log_p += np.log(1 - theta)
            log_probs[c] = log_p
        preds[n] = np.argmax(log_probs)
    return preds

def train_nb(train_feats, train_class, s=5):
    """naive bayes with back-off smoothing: all parent_feat = -1"""
    parent_feat = [-1] * train_feats.shape[1]
    return train_tan(train_feats, train_class, parent_feat, s)

#===== run experiments =====
np.random.seed(42)

#withhold 183 records as test set
N = len(classes)
perm = np.random.permutation(N)
test_idx = perm[:183]
remaining_idx = perm[183:]

test_feats = features[test_idx]
test_class = classes[test_idx]

#learn structure on full training data first for the writeup
full_train_feats = features[remaining_idx]
full_train_class = classes[remaining_idx]
parent_feat_full, tree_edges_full, weights_full = learn_tan_structure(
   full_train_feats, full_train_class, n_feat)

print("\n=== TAN Structure (full training data) ===")
print(f"Feature parent assignments (feature index -> parent feature index)
   :")
for i in range(n_feat):
    pa = parent_feat_full[i]
    if pa == -1:
        print(f"  X{i+1} ({feat_names[i]}): root (no feature parent)")
    else:
        print(f"  X{i+1} ({feat_names[i]}): parent = X{pa+1} ({feat_names[
            pa]})")

print(f"\nTree edges with CMI weights:")
for i, j, w in sorted(tree_edges_full, key=lambda x: -x[2]):
    print(f"  X{i+1}-X{j+1}: I = {w:.6f}")

#classification experiments
train_sizes = [100, 200, 300, 400, 500]
nb_errors = []
tan_errors = []

for m in train_sizes:
    train_idx = remaining_idx[:m]
    tr_feats = features[train_idx]
    tr_class = classes[train_idx]

    #TAN
```

```
    parent_feat_m, _, _ = learn_tan_structure(tr_feats, tr_class, n_feat)
    pc_tan, cpts_tan = train_tan(tr_feats, tr_class, parent_feat_m, s=5)
    preds_tan = predict_tan(test_feats, pc_tan, cpts_tan, parent_feat_m,
        n_feat)
    err_tan = np.mean(preds_tan != test_class)
    tan_errors.append(err_tan)

    #Naive Bayes
    parent_nb = [-1] * n_feat
    pc_nb, cpts_nb = train_nb(tr_feats, tr_class, s=5)
    preds_nb = predict_tan(test_feats, pc_nb, cpts_nb, parent_nb, n_feat)
    err_nb = np.mean(preds_nb != test_class)
    nb_errors.append(err_nb)

    print(f"m={m}: NB error={err_nb:.4f}, TAN error={err_tan:.4f}")

#plot
plt.figure(figsize=(8, 5))
plt.plot(train_sizes, nb_errors, 'bo-', label='Naive Bayes', linewidth=2)
plt.plot(train_sizes, tan_errors, 'rs-', label='TAN', linewidth=2)
plt.xlabel('Training set size (m)', fontsize=12)
plt.ylabel('Classification error', fontsize=12)
plt.title('NB vs TAN Classification Error on Breast Cancer Data', fontsize
    =13)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(SCRIPT_DIR, 'figures/tan_vs_nb_error.png'), dpi
    =150, bbox_inches='tight')
print(f"\nSaved {os.path.join(SCRIPT_DIR, 'figures/tan_vs_nb_error.png')}"
    )

#print CMI weight matrix
print("\n=== CMI Weight Matrix ===")
for i in range(n_feat):
    row = []
    for j in range(n_feat):
        row.append(f"{weights_full[i,j]:.4f}")
    print(f"  X{i+1}: { '  '.join(row)}")
```

Problem 5 (Printer Nightmare – EM with Missing Data)

BN Structure



The five root nodes (Fuse, Drum, Toner, Paper, Roller) are possible diagnoses. The five leaf nodes are observable faults. All variables are binary. The data has 15 datapoints with many missing entries (coded as NaN).

EM Algorithm

Since the data has missing values, we cannot compute MLEs by counting. Instead we use EM:

E-step: For each datapoint, enumerate all possible completions of missing variables (2^k if k variables are missing). Weight each completion by its probability under the current CPTs (normalized to sum to 1). Accumulate expected sufficient statistics (counts of each variable-value conditioned on parent configuration).

M-step: Update each CPT entry as the ratio of expected counts:

$$\theta_v^{\text{new}}(\text{pa_config}) = \frac{E[\text{count}(v{=}1, \text{pa_config})]}{E[\text{count}(v{=}0, \text{pa_config})] + E[\text{count}(v{=}1, \text{pa_config})]}$$

I ran EM with 5 random restarts, iterating until the maximum CPT change falls below 10^{-8}. All restarts converged to similar log-likelihood values (≈ -49.05), suggesting a stable solution.

Learned Priors

Diagnosis	$P(\text{problem} = 1)$
Fuse assembly malfunction	0.2173
Drum unit problem	0.3869
Toner out	0.4452
Poor paper quality	0.6339
Worn roller	0.2821

Poor paper quality is the most common diagnosis, consistent with the problem description that Cheapco uses substandard materials.

Query Result

Given evidence: no wrinkled pages (Wrinkled = 0), no burning smell (Burning = 0), and poor print quality (Quality = 1), we compute $P(\text{Drum} = 1 \mid \text{evidence})$ by summing over all non-evidence variables weighted by the learned CPTs.

$$P(\text{Drum unit problem} = 1 \mid \text{no wrinkled, no burning, poor quality}) = \mathbf{0.7564}$$

This makes sense: poor print quality without burning smell rules out fuse problems ($P(\text{Burning} = 0 \mid \text{Fuse} = 1) \approx 0$, so fuse is almost certainly fine). The drum unit and toner are the main remaining

causes of poor quality. The no-wrinkled-pages evidence further narrows the diagnosis toward the drum, since wrinkled pages would be expected if paper quality were the primary issue.

Code:

```
import numpy as np
import os
import scipy.io as sio

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

#load data: states 1,2,nan -> 0,1,missing
data = sio.loadmat(os.path.join(SCRIPT_DIR, 'EMprinter.mat'))
X = data['x']  #shape (10, 15): 10 variables, 15 datapoints
print(f"Data␣shape:␣{X.shape}")
print(f"Variables␣x␣Datapoints")

#remap: 1->0, 2->1, nan->-1
D = np.full_like(X, -1, dtype=float)
D[X == 1] = 0
D[X == 2] = 1
#nan stays as -1
D[np.isnan(X)] = -1
N = D.shape[1]  #15 datapoints
print(f"N␣=␣{N}␣datapoints")

#BN structure (from figure):
#0: Fuse (root)
#1: Drum (root)
#2: Toner (root)
#3: Paper (root)
#4: Roller (root)
#5: Burning -> parents: {Fuse}
#6: Quality -> parents: {Fuse, Drum, Toner}
#7: Wrinkled -> parents: {Fuse, Drum, Paper}
#8: MultPages -> parents: {Paper, Roller}
#9: PaperJam -> parents: {Drum, Toner, Paper, Roller}

var_names = ['Fuse', 'Drum', 'Toner', 'Paper', 'Roller',
             'Burning', 'Quality', 'Wrinkled', 'MultPages', 'PaperJam']

parents = {
    0: [],
    1: [],
    2: [],
    3: [],
    4: [],
    5: [0],
    6: [0, 1, 2],
    7: [0, 1, 3],
    8: [3, 4],
    9: [1, 2, 3, 4],
}

n_vars = 10
```

```
def init_cpts():
    """initialize␣CPTs␣randomly␣but␣valid"""
    cpts = {}
    for v in range(n_vars):
        pa = parents[v]
        n_pa_configs = 2 ** len(pa)
        #P(v=1 | pa_config) for each parent configuration
        #initialize near 0.5 with small noise
        cpts[v] = 0.3 + 0.4 * np.random.rand(n_pa_configs)
    return cpts

def pa_config_index(assignment, pa_list):
    """convert␣parent␣assignment␣to␣index␣(binary␣encoding)"""
    idx = 0
    for k, p in enumerate(pa_list):
        idx += int(assignment[p]) * (2 ** k)
    return idx

def log_prob_sample(assignment, cpts):
    """log␣P(assignment)␣given␣CPTs.␣assignment␣is␣length-10␣array␣of␣0/1"
       ""
    lp = 0.0
    for v in range(n_vars):
        pa = parents[v]
        pa_idx = pa_config_index(assignment, pa)
        theta = cpts[v][pa_idx]  #P(v=1|pa)
        if assignment[v] == 1:
            lp += np.log(theta + 1e-300)
        else:
            lp += np.log(1 - theta + 1e-300)
    return lp

def em_learn(D, max_iter=200, tol=1e-6, seed=0):
    """EM␣algorithm␣for␣BN␣with␣missing␣data"""
    np.random.seed(seed)
    cpts = init_cpts()
    N = D.shape[1]

    for iteration in range(max_iter):
        #E-step: for each datapoint, compute posterior over missing
           variables
        #by enumerating all completions of missing values
        expected_counts = {}
        for v in range(n_vars):
            pa = parents[v]
            n_pa_configs = 2 ** len(pa)
            #counts[pa_config, v_val]
            expected_counts[v] = np.zeros((n_pa_configs, 2))

        total_ll = 0.0

        for n in range(N):
            obs = D[:, n]
```

```
        missing = np.where(obs == -1)[0]
        observed = np.where(obs != -1)[0]
        n_missing = len(missing)

        if n_missing == 0:
            #fully observed
            assignment = obs.astype(int)
            lp = log_prob_sample(assignment, cpts)
            total_ll += lp
            for v in range(n_vars):
                pa = parents[v]
                pa_idx = pa_config_index(assignment, pa)
                expected_counts[v][pa_idx, assignment[v]] += 1.0
        else:
            #enumerate all 2^n_missing completions
            log_probs = []
            completions = []
            for combo in range(2 ** n_missing):
                assignment = obs.copy().astype(int)
                for k, m_idx in enumerate(missing):
                    assignment[m_idx] = (combo >> k) & 1
                lp = log_prob_sample(assignment, cpts)
                log_probs.append(lp)
                completions.append(assignment.copy())

            #normalize
            log_probs = np.array(log_probs)
            max_lp = np.max(log_probs)
            weights = np.exp(log_probs - max_lp)
            weights /= np.sum(weights)
            total_ll += max_lp + np.log(np.sum(np.exp(log_probs -
               max_lp)))

            for w, assignment in zip(weights, completions):
                for v in range(n_vars):
                    pa = parents[v]
                    pa_idx = pa_config_index(assignment, pa)
                    expected_counts[v][pa_idx, assignment[v]] += w

    #M-step: update CPTs from expected counts
    old_cpts = {v: cpts[v].copy() for v in range(n_vars)}
    for v in range(n_vars):
        n_pa_configs = expected_counts[v].shape[0]
        for pa_idx in range(n_pa_configs):
            total = expected_counts[v][pa_idx, 0] + expected_counts[v
               ][pa_idx, 1]
            if total > 1e-10:
                cpts[v][pa_idx] = expected_counts[v][pa_idx, 1] /
                   total
            else:
                cpts[v][pa_idx] = 0.5

    #check convergence
    max_change = 0
```

```
        for v in range(n_vars):
            max_change = max(max_change, np.max(np.abs(cpts[v] - old_cpts[
               v])))

        if iteration % 20 == 0 or max_change < tol:
            print(f"␣␣iter␣{iteration}:␣LL={total_ll:.4f},␣max_change={
               max_change:.8f}")

        if max_change < tol:
            print(f"␣␣converged␣at␣iteration␣{iteration}")
            break

    return cpts, total_ll

#run EM with multiple restarts
best_ll = -np.inf
best_cpts = None
n_restarts = 5

for seed in range(n_restarts):
    print(f"\n---␣EM␣restart␣{seed+1}␣---")
    cpts, ll = em_learn(D, max_iter=500, tol=1e-8, seed=seed)
    print(f"␣␣final␣LL␣=␣{ll:.4f}")
    if ll > best_ll:
        best_ll = ll
        best_cpts = cpts

print(f"\nBest␣LL␣across␣restarts:␣{best_ll:.4f}")

#print learned CPTs
print("\n===␣Learned␣CPTs␣===")
for v in range(n_vars):
    pa = parents[v]
    print(f"\n{var_names[v]}␣(parents:␣{[var_names[p]␣for␣p␣in␣pa]}):")
    n_configs = 2 ** len(pa)
    for pa_idx in range(n_configs):
        #decode parent config
        pa_vals = {}
        for k, p in enumerate(pa):
            pa_vals[var_names[p]] = (pa_idx >> k) & 1
        theta = best_cpts[v][pa_idx]
        if len(pa) == 0:
            print(f"␣␣P({var_names[v]}=1)␣=␣{theta:.4f}")
        else:
            pa_str = ',␣'.join([f"{k}={v}" for k, v in pa_vals.items()])
            print(f"␣␣P({var_names[v]}=1␣|␣{pa_str})␣=␣{theta:.4f}")

#query: P(Drum=1 | Wrinkled=0, Burning=0, Quality=1)
#evidence: var 7=0, var 5=0, var 6=1
print("\n===␣Query:␣P(Drum=1␣|␣Wrinkled=0,␣Burning=0,␣Quality=1)␣===")

evidence = {5: 0, 6: 1, 7: 0}  #burning=0, quality=1 (poor), wrinkled=0
query_var = 1  #drum
```

```
#enumerate all non-evidence variables' values
non_evidence = [v for v in range(n_vars) if v not in evidence]
n_free = len(non_evidence)

log_probs_drum0 = []
log_probs_drum1 = []

for combo in range(2 ** n_free):
    assignment = np.zeros(n_vars, dtype=int)
    #set evidence
    for v, val in evidence.items():
        assignment[v] = val
    #set free variables
    for k, v in enumerate(non_evidence):
        assignment[v] = (combo >> k) & 1

    lp = log_prob_sample(assignment, best_cpts)

    if assignment[query_var] == 0:
        log_probs_drum0.append(lp)
    else:
        log_probs_drum1.append(lp)

#compute P(drum=1 | evidence)
log_probs_drum0 = np.array(log_probs_drum0)
log_probs_drum1 = np.array(log_probs_drum1)

max_all = max(np.max(log_probs_drum0), np.max(log_probs_drum1))
p0 = np.sum(np.exp(log_probs_drum0 - max_all))
p1 = np.sum(np.exp(log_probs_drum1 - max_all))

prob_drum1 = p1 / (p0 + p1)
prob_drum0 = p0 / (p0 + p1)

print(f"P(Drum=0 | evidence) = {prob_drum0:.4f}")
print(f"P(Drum=1 | evidence) = {prob_drum1:.4f}")
```